本文档由 funstory.ai 的开源 PDF 翻译库 BabelDOC v0.6.4 (https://github.com/funstory-ai/BabelDOC) 翻译，本仓库正在积极的建设当中，欢迎 star 和关注。

g²o: 一个通用的图优化框架

Rainer K¨ummerle Giorgio Grisetti HaukeStrasdat Kurt Konolige Wolfram Burgard

摘要—许多机器人学和计算机视觉中的流行问题，包括各种类型的同步定位与地图构建（SLAM）或束调整（BA），都可以表述为对误差函数的最小二乘优化，该误差函数可以表示为图。本文描述了这类问题的通用结构，并介绍了g²o，一个用于优化基于图的非线性误差函数的开源C++ 框架。我们的系统设计为易于扩展到广泛的问题，新问题通常可以用几行代码指定。当前实现为几种 SLAM 和 BA 变体提供了解决方案。我们在多种真实世界和模拟数据集上进行了评估。结果表明，虽然 g²o 具有通用性，但其性能与针对特定问题最先进方法实现的性能相当。

I. 引言

机器人学以及计算机视觉中的许多问题都涉及最小化一个可以表示为图的非线性误差函数。典型实例包括同时定位与地图构建（SLAM）
[19], [5], [22], [10], [16], [26] 或束调整（BA）
[27], [15], [18]。这些问题中的总体目标是在受高斯噪声影响的测量集中找到能最大程度解释参数或状态变量的配置。例如，在基于图的SLAM中，状态变量可以是机器人环境中的位置，也可以是机器人传感器可观察的地图中关键点的位置。因此，一个测量值仅取决于两个状态变量的相对位置，例如，两个连续姿态之间的里程计测量值仅取决于连接的姿态。类似地，在束调整或基于地标的SLAM中，一个3D点或关键点的测量值仅取决于观察点在世界中的位置以及传感器的位置。

所有这些问题都可以表示为图。其中，图的每个节点代表一个需要优化的状态变量，每条连接两个变量的边则代表这两个节点之间的成对观测。在文献中，已经提出了许多方法来处理这类问题。使用高斯-牛顿、勒文贝格-马夸特（LM）、高斯-赛德尔松弛或梯度下降变体的标准方法进行简单实现，通常可以为大多数应用提供可接受的结果。

这项工作部分得到了欧洲委员会的支持，包括FP7-231888-EUROPA、FP7-248873-RADHAR项目，以及欧洲研究委员会启动资助210346。R. K¨ummerle、G. Grisetti和W. Burgard隶属于弗莱堡大学。G. Grisetti同时隶属于罗马大学Sapienza。H. Strasdat隶属于伦敦帝国理工学院计算机系。K. Konolige隶属于Willow Garage，并担任斯坦福大学客座教授。

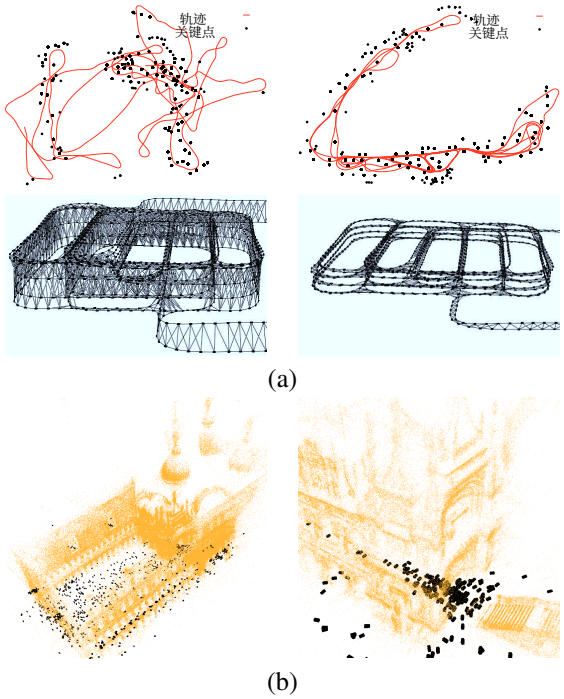


图1。使用我们的系统处理的真实世界数据集：(a)的第一行显示了维多利亚公园数据集，该数据集包含2D里程计和2D关键点测量。 (a)的第二行描绘了一个多层车库的3D位姿图。左侧图像显示初始状态，右侧列描绘了相应的优化过程结果。 (b)展示了经过我们系统优化的威尼斯束调整数据集的全视图和缩放视图。该数据集包含871个相机位姿和2,838,740个投影。

然而，要实现最佳性能，则需要大量的努力和领域知识。

在本文中，我们描述了一个通用的框架，用于执行可以表示为图的非线性最小二乘问题的优化。我们称此框架为g²o（“通用图优化”）。图1概述了可以使用g²o作为优化后端解决的问题的各种类型。所提出的系统实现了与最先进算法的实现相当的性能，同时能够接受非线性测量的通用形式。我们通过利用算法来实现效率，

- 利用图的稀疏连接性，
- 利用上述问题中经常出现的图的特殊结构，
- 使用高级方法来解决稀疏线性系统，
- 并利用现代处理器的特性，如SIMD指令，并优化缓存使用。

尽管高效，g²o 具有高度通用性和可扩展性：

一个 2D SLAM 算法可以用不到 30 行 C++ 代码实现。用户只需指定误差函数及其参数。

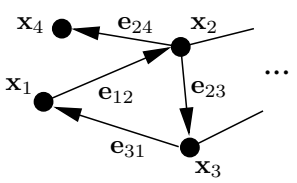
在本文中，我们应用 g²o 到不同类别的最小二乘优化问题，并将其性能与问题特定算法的不同实现进行比较。我们展示了在大量真实世界和模拟数据集上进行的评估；在所有实验中，g²o 提供了与最先进方法相当的性能，在几种情况下甚至超越了它们。

本文的其余部分组织如下。我们首先讨论相关工作，特别关注 SLAM 和束调整问题的解决方案。随后，在第三节中，我们描述了我们的系统解决的图嵌入优化问题，并讨论通过高斯-牛顿或 LM 进行的非线性最小二乘。在第四节中，我们讨论了我们的实现提供的功能。最后，在第五节中，我们展示了 g²o 的广泛实验评估，并将其与其他最先进的问题特定方法进行比较。

II. 相关工作

过去，图优化问题在机器人学和计算机视觉领域得到了深入研究。一项开创性工作是 Lu 和 Milios [19] 的研究，其中通过扫描匹配测量了两次扫描之间的相对运动，并通过迭代线性化优化了生成的图。虽然当时，图的优化被认为对于实时性能过于耗时，但直接线性求解器（例如， [4]）的最新发展使得基于图的 SLAM 重新流行起来，并且提出了大量通过图优化解决 SLAM 的不同方法。例如，Howard 等人 [12] 应用松弛来构建地图。Duckett 等人 [6] 提出使用高斯-赛德尔松弛来最小化约束网络中的误差。Frese 等人 [8] 引入了多级松弛（MLR），这是高斯-赛德尔松弛的一个变体，它在不同分辨率级别应用松弛。最近，Olson 等人 [22] 提出了一种梯度下降方法来优化位姿图。后来，Grisetti 等人 [10] 通过应用基于树的参数化扩展了这种方法，从而提高了收敛速度。这两种方法对初始猜测都很鲁棒，而且相对容易实现。然而，它们假设协方差大致是球形的，因此在优化位姿图时存在困难，其中某些约束的协方差与零空间或特征值有显著差异。

图优化可以被视为一个非线性最小二乘问题，通常通过在当前状态周围形成线性系统、求解并迭代来解决。解决线性系统的一种有前景的技术是预条件共轭梯度（PCG），Konolige [17] 以及 Montemerlo 和 Thrun [20] 都曾将其用作高效的求解器来处理大型稀疏位姿约束系统。由于它在某些问题上具有很高的效率，g²o



$$
\begin{aligned}
F(x) = {} & e_{12}^{\top}\,\Omega_{12}\,e_{12} \\
& + \mathbf{e}_{23}^{\top}\,\Omega_{23}\mathbf{e}_{23} \\
& + \mathbf{e}_{31}^{\top}\,\Omega_{31}\,e_{31} \\
& + \mathbf{e}_{24}^{\top}\,\Omega_{24}\,e_{24} \\
& + \dots
\end{aligned}
$$

图2。本示例说明了如何用图来表示目标函数。

包括一个稀疏 PCG 求解器的实现，该求解器应用了块-Jacobi 预处理器 [13]。

近年来，Dellaert及其同事提出了一种名为√SAM [5] 的系统，他们使用稀疏直接线性求解器 [4]来实现。Kaess等人 [14] 引入了该系统的一个变体iSAM，它能够更新非线性最小二乘问题相关的线性矩阵。Kono-lige等人 [16] 展示了如何通过利用线性系统典型的稀疏结构来高效地构造线性矩阵。然而，后一种方法仅限于2D位姿图。在**g²o** 中，我们的系统与这些系统有相似之处。我们的系统可以应用于SLAM和BA优化问题的所有变体，例如带关键点的2D SLAM、使用单目相机的BA或使用立体视觉的BA。然而，**g²o**在所有我们用于评估目的的数据上，展示了相比这些系统大幅提升的性能。

在计算机视觉中，稀疏束调整 [27] 是一种非线性最小二乘法，它利用了点与相机位姿之间雅可比矩阵模式的稀疏性。最近，出现了几个系统 [15],[13] ，它们推进了稀疏线性求解器类似的概念以及针对大型系统（~100M 个稀疏矩阵元素）的舒尔约化（参见第 III-D 节）的高效计算。还有一些基于非线性共轭梯度的新系统，它们从不显式形成线性系统 [1], [2]；这些收敛速度较慢，但可以处理极其庞大的数据集（~1000M 个矩阵元素）。在本文中，我们将 g²o 与 [15], 的 SSBA 系统进行比较，后者是目前公开可用的性能最佳的系统。

III. 使用最小二乘法的非线性图优化

机器人学或计算机视觉中的许多问题都可以通过找到形如该函数的最小值来解决：

$$
\mathbf{F}(\mathbf{x}) = \sum_{\langle i,j\rangle\in\mathcal{C}} \underbrace{\mathbf{e}(\mathbf{x}_i,\mathbf{x}_j,\mathbf{z}_{ij})^{\top}\boldsymbol{\Omega}_{ij}\mathbf{e}(\mathbf{x}_i,\mathbf{x}_j,\mathbf{z}_{ij})}_{\mathbf{F}_{ij}} \quad (1)
$$

$$
\mathbf{x}^{*} = \operatorname*{argmin}_{\mathbf{x}} \mathbf{F}(\mathbf{x}). \quad (2)
$$

这里，$x = (x_1^{\top}, \dots, x_n^{\top})^{\top}$ 是一个参数向量，其中每个 x_i 代表一个通用参数块，z_{ij}和 Ω_{ij} 分别表示与参数 x_j 和 x_i相关的约束的均值和信息矩阵，而 $e(x_i,x_j,z_{ij})$ 是一个误差函数向量，用于衡量参数块 x_i 和 x_j 满足约束 z_{ij} 的程度。当 x_i 和 x_j 完美匹配约束时，它为 0。

为简化符号表示，在本文的其余部分，我们将测量值编码到误差函数的索引中：

$$\mathbf{e}(\mathbf{x}_i, \mathbf{x}_j, \mathbf{z}_{ij}) \overset{\text{def.}}{=} \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j) \overset{\text{def.}}{=} \mathbf{e}_{ij}(\mathbf{x}). \tag{3}$$

请注意，每个误差函数、每个参数块以及每个误差函数可以跨越不同的空间。以这种形式的问题可以有效地用有向图表示。图中的节点 i 代表参数块 x_i ，节点 i和 j 之间的边代表两个参数块 x_i 和 $\mathrm{x}j$ 之间的有序约束。图2展示了一个图与目标函数之间的映射示例。

A. 最小二乘优化

如果已知参数的良好初始猜测 $\breve{}\mathrm{x}$，则可以通过使用流行的Gauss-Newton或LM算法 [23, §15.5] 来获得方程(2) 的数值解。其思想是在当前初始猜测 $\breve{}\mathrm{x}$ 附近用误差函数的一阶泰勒展开式来近似误差函数。

$$\begin{aligned}\mathbf{e}_{ij}(\breve{\mathbf{x}}_i + \boldsymbol{\Delta}\mathbf{x}_i, \breve{\mathbf{x}}_j + \boldsymbol{\Delta}\mathbf{x}_j) &= \mathbf{e}_{ij}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}) &(4)\\ &\simeq \mathbf{e}_{ij} + \mathbf{J}_{ij}\boldsymbol{\Delta}\mathbf{x}. &(5)\end{aligned}$$

Here，$\mathrm{J}ij$ 是$\mathrm{e}ij$(x)在 $\breve{}\mathrm{x}$中计算的雅可比矩阵，$\mathrm{e}ij$ 定义了$\mathrm{e}=(\breve{}\mathrm{x})$。将公式(5)代入公式(1)的误差项$\mathrm{F}ij$ 中，我们得到

$$\begin{aligned}&\mathbf{F}_{ij}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}) &(6)\\ &= \mathbf{e}_{ij}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x})^\top \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}) &(7)\\ &\simeq (\mathbf{e}_{ij} + \mathbf{J}_{ij}\boldsymbol{\Delta}\mathbf{x})^\top \boldsymbol{\Omega}_{ij} (\mathbf{e}_{ij} + \mathbf{J}_{ij}\boldsymbol{\Delta}\mathbf{x}) &(8)\\ &= \underbrace{\mathbf{e}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij}}_{\mathrm{c}_{ij}} + 2\underbrace{\mathbf{e}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{J}_{ij}}_{\mathbf{b}_{ij}} \boldsymbol{\Delta}\mathbf{x} + \boldsymbol{\Delta}\mathbf{x}^\top \underbrace{\mathbf{J}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{J}_{ij}}_{\mathbf{H}_{ij}} \boldsymbol{\Delta}\mathbf{x} &(9)\\ &= \mathrm{c}_{ij} + 2\mathbf{b}_{ij}\boldsymbol{\Delta}\mathbf{x} + \boldsymbol{\Delta}\mathbf{x}^\top \mathbf{H}_{ij} \boldsymbol{\Delta}\mathbf{x} &(10)\end{aligned}$$

基于此局部近似，我们可以将式(1)中给出的函数F(x)改写为

$$\begin{aligned}\mathbf{F}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}) &= \sum_{\langle i,j\rangle \in \mathcal{C}} \mathbf{F}_{ij}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}) &(11)\\ &\simeq \sum_{\langle i,j\rangle \in \mathcal{C}} \mathrm{c}_{ij} + 2\mathbf{b}_{ij}\boldsymbol{\Delta}\mathbf{x} + \boldsymbol{\Delta}\mathbf{x}^\top \mathbf{H}_{ij} \boldsymbol{\Delta}\mathbf{x} &(12)\\ &= \mathrm{c} + 2\mathbf{b}^\top \boldsymbol{\Delta}\mathbf{x} + \boldsymbol{\Delta}\mathbf{x}^\top \mathbf{H} \boldsymbol{\Delta}\mathbf{x}. &(13)\end{aligned}$$

式(13)中的二次型是通过将式(12)中的$\mathrm{c} = \sum \mathrm{c}ij$、$\mathrm{b} = \sum \mathrm{b}ij$ 和$\mathrm{H} = \sum \mathrm{H}ij$进行设置得到的。它可以通过求解线性系统在{x}_ $\boldsymbol{\Delta}$处进行最小化

$$\mathbf{H}\,\boldsymbol{\Delta}\mathbf{x}^* = -\mathbf{b}. \tag{14}$$

在此处，H是系统的信息矩阵。通过将增量{x}_ $\boldsymbol{\Delta}\mathrm{x}^*$ 添加到初始猜测值，即可得到解

$$\mathbf{x}^* = \breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}^*. \tag{15}$$

流行的Gauss-Newton算法迭代公式(13)中的线性化、公式(14)中的解以及公式(15)中的更新步骤。在每次迭代中，先前解被用作线性化点和初始猜测，直到满足给定的终止条件。

LM算法引入了阻尼因子和后备操作来控制Gauss-Newton的收敛性。它不是求解公式14，而是求解一个阻尼版本

$$(\mathbf{H} + \lambda\mathbf{I})\,\boldsymbol{\Delta}\mathbf{x}^* = -\mathbf{b}. \tag{16}$$

在此处 λ 是一个阻尼因子： λ 越高，$\boldsymbol{\Delta}$x就越小。这在处理非线性曲面时用于控制步长。LM算法背后的思想是动态控制阻尼因子。在每次迭代中都会监控新配置的误差。如果新误差低于前一个误差， λ 将在下一次迭代中减小。否则，解决方案将被撤销，lambda将增加。有关我们框架中实现的LM算法的更详细说明，我们参考了 [18]。

B. 替代参数化

上述描述的流程是多元函数最小化的通用方法。它们假设参数空间 x 是欧几里得空间，这对于 SLAM 或束调整等若干问题并不适用。为了处理跨越非欧几里得空间的状态变量，常见的方法是在不同于参数空间 x_i 的空间中表达增量 $\boldsymbol{\Delta}\mathrm{x}_i$ 。

例如，在 SLAM 问题的背景下，每个参数块 x_i 包含一个平移向量 t_i 和一个旋转分量 α_i。平移 t_i 显然形成了一个欧几里得空间。相比之下，旋转分量 α_i 跨越非欧几里得二维或三维旋转群 $SO(2)$ 或 $SO(3)$。为了避免奇点，这些空间通常以超参数化的方式描述，例如通过旋转矩阵或四元数。直接将这些超参数化表示应用于公式 (15) 会破坏超参数化引入的约束。为了克服这个问题，可以使用旋转的最小表示（例如 3D 中的欧拉角）。然而，这会引入奇点。

另一种思路是计算一个新的误差函数，其中 $\boldsymbol{\Delta}\mathrm{x}_i$ 是围绕当前变量$\breve{}\mathrm{x}_i$的扰动。 $\boldsymbol{\Delta}\mathrm{x}_i$ 使用旋转的最小表示，而x_i 则采用过参数化的表示。由于 $\boldsymbol{\Delta}\mathrm{x}_i$通常很小，它们远离奇点。优化后变量$\mathrm{x}_i^*$ 的新值可以通过应用非线性算子 $\boxplus$ 的增量来获得：$\mathrm{Dom}(\mathrm{x}_i) \times \mathrm{Dom}(\boldsymbol{\Delta}\mathrm{x}_i) \to \mathrm{Dom}(\mathrm{x}_i)$，如下所示：

$$\mathbf{x}_i^* = \breve{\mathbf{x}}_i \boxplus \boldsymbol{\Delta}\mathbf{x}_i^*. \tag{17}$$

例如，在3D SLAM中，可以将增量 $\boldsymbol{\Delta}\mathrm{x}_i$ 表示为平移向量和归一化四元数的轴。姿态 x_i 表示为平移向量和完整四元数。 $\boxplus$ 运算符通过将增量转换为与状态变量相同的表示形式后，使用标准运动组合运算符 $\oplus$ （参见 [25]）将增量 $\boldsymbol{\Delta}\mathrm{x}_i$ 应用于 x_i ：

$$\breve{\mathbf{x}}_i \boxplus \boldsymbol{\Delta}\mathbf{x}_i^* \overset{\text{def.}}{=} \breve{\mathbf{x}}_i \oplus \boldsymbol{\Delta}\mathbf{x}_i^*. \tag{18}$$

使用此运算符，可以定义一个新的误差函数

$$\mathbf{e}_{ij}(\boldsymbol{\Delta x}_i, \boldsymbol{\Delta x}_j) \overset{\text{def.}}{=} \mathbf{e}_{ij}(\breve{\mathbf{x}}_i \boxplus \boldsymbol{\Delta x}_i, \breve{\mathbf{x}}_j \boxplus \boldsymbol{\Delta x}_j) \tag{19}$$

$$= \mathbf{e}_{ij}(\breve{\mathbf{x}} \boxplus \boldsymbol{\Delta x}) \simeq \mathbf{e}_{ij} + \mathbf{J}_{ij}\boldsymbol{\Delta x}, \tag{20}$$

在 $\breve{\mathbf{x}}$ 跨越原始过参数化空间的情况下。雅可比矩阵 $\mathbf{J}_{ij}$ 变为

$$\mathbf{J}_{ij} = \left.\frac{\partial \mathbf{e}_{ij}(\breve{\mathbf{x}} \boxplus \boldsymbol{\Delta x})}{\partial \boldsymbol{\Delta x}}\right|_{\boldsymbol{\Delta x}=\mathbf{0}}. \tag{21}$$

由于增量 $\boldsymbol{\Delta}\tilde{\mathbf{x}}^*$ 是在初始猜测 $\breve{\mathbf{x}}$ 的局部欧几里得环境中计算的，因此需要通过 $\boxplus$ 算子将它们重新映射到原始冗余空间中。

我们的框架允许轻松定义增量状态变量的不同空间，从而在同一个问题中透明地支持任意参数化。无论选择何种参数化，黑塞矩阵 H 的结构通常都得到保留。

C. 线性化系统的结构

根据公式 (13)，矩阵 H 和向量 b 是通过对每个约束的一组矩阵和向量求和得到的。如果我们设置 $\mathbf{b}_{ij} = \mathbf{J}_{ij}^\top \Omega_{ij}\mathbf{e}_{ij}$ 和 $\mathbf{H}_{ij} = \mathbf{J}_{ij}^\top \Omega_{ij}\mathbf{J}_{ij}$，我们可以将 b 和 H 重写为

$$\mathbf{b} = \sum_{\langle i,j\rangle \in \mathcal{C}} \mathbf{b}_{ij} \qquad \mathbf{H} = \sum_{\langle i,j\rangle \in \mathcal{C}} \mathbf{H}_{ij}. \tag{22}$$

每个约束都将向系统贡献一个附加项。这个附加项的结构取决于误差函数的雅可比矩阵。由于约束的误差函数仅取决于两个节点的值，公式 (5) 中的雅可比矩阵具有以下形式：

$$\mathbf{J}_{ij} = \begin{pmatrix} \mathbf{0}\cdots\mathbf{0} & \underbrace{\mathbf{A}_{ij}}_{i} & \mathbf{0}\cdots\mathbf{0} & \underbrace{\mathbf{B}_{ij}}_{j} & \mathbf{0}\cdots\mathbf{0} \end{pmatrix}. \tag{23}$$

这里 $\mathbf{A}_{ij}$ 和 $\mathbf{B}_{ij}$ 是误差函数对 $\boldsymbol{\Delta x}_i$ 和 $\boldsymbol{\Delta x}_j$的导数。根据公式 (9)，我们得到块矩阵 $\mathbf{H}_{ij}$ 的以下结构：

$$\mathbf{H}_{ij} = \begin{pmatrix} \ddots & & \\ & \mathbf{A}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{A}_{ij} & \cdots & \mathbf{A}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{B}_{ij} \\ & \vdots & & \vdots \\ & \mathbf{B}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{A}_{ij} & \cdots & \mathbf{B}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{B}_{ij} \\ & & & \ddots \end{pmatrix} \mathbf{b}_{ij} = \begin{pmatrix} \vdots \\ \mathbf{A}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij} \\ \vdots \\ \mathbf{B}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij} \\ \vdots \end{pmatrix}$$

为简化符号表示，我们省略了零块。读者可能会注意到，矩阵H的块结构是图的邻接矩阵。因此，它具有与图中边数成正比的非零块数量。这通常导致稀疏H矩阵。在Ing²o中，我们利用H的这一特性，采用最先进的方法求解公式(14)的线性系统。

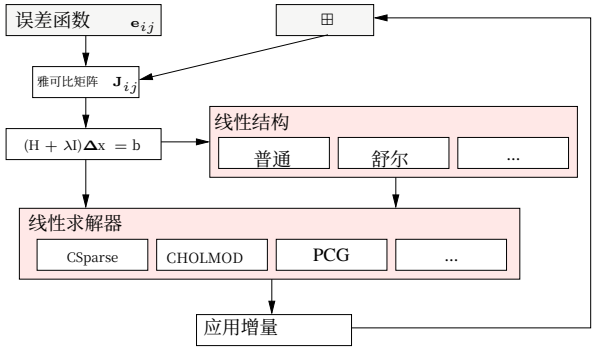


图3。我们框架的概述。对于解决一个新的优化问题，只需要指定灰色的方框。此外，该框架允许添加不同的线性求解器。

D. 具有特殊结构的系统

某些问题（例如，BA）会导致H矩阵具有更典型的结构。我们的系统可以利用这些特殊结构来提高性能。在BA中，通常有两种类型的变量，即相机位姿p和相机观测到的关键点位姿l。通过重新排列公式(14)中的变量，使相机位姿具有较低索引，我们得到系统

$$\begin{pmatrix} \mathbf{H}_{\mathbf{pp}} & \mathbf{H}_{\mathbf{pl}} \\ \mathbf{H}_{\mathbf{pl}}^\top & \mathbf{H}_{\mathbf{ll}} \end{pmatrix} \begin{pmatrix} \boldsymbol{\Delta x}_{\mathbf{p}}^* \\ \boldsymbol{\Delta x}_{\mathbf{l}}^* \end{pmatrix} = \begin{pmatrix} -\mathbf{b}_{\mathbf{p}} \\ -\mathbf{b}_{\mathbf{l}} \end{pmatrix}. \tag{24}$$

可以证明，通过取H矩阵的Schur补 [7]，可以形成等效的简化系统

$$\left(\mathbf{H}_{\mathbf{pp}} - \mathbf{H}_{\mathbf{pl}}\mathbf{H}_{\mathbf{ll}}^{-1}\mathbf{H}_{\mathbf{pl}}^\top\right) \boldsymbol{\Delta x}_{\mathbf{p}}^* = -\mathbf{b}_{\mathbf{p}} + \mathbf{H}_{\mathbf{pl}}\mathbf{H}_{\mathbf{ll}}^{-1}\mathbf{b}_{\mathbf{l}}. \tag{25}$$

注意，计算H^{-1} ll 很简单，因为H_{ll} 是一个块对角矩阵。求解公式(25)可以得到相机姿态的增量 $\boldsymbol{\Delta x}_{\mathbf{p}}^*$，利用这个结果我们可以求解

$$\mathbf{H}_{\mathbf{ll}}\boldsymbol{\Delta x}_{\mathbf{l}}^* = -\mathbf{b}_{\mathbf{l}} - \mathbf{H}_{\mathbf{pl}}^\top \boldsymbol{\Delta x}_{\mathbf{p}}^*, \tag{26}$$

这会导致 $\boldsymbol{\Delta x}_{\mathbf{l}}^*$，用于调整观测到的世界特征。通常世界特征的数量多于相机位姿，因此尽管在计算公式(25)左侧矩阵上花费了额外的时间，但公式(25)的求解速度仍然比公式(14)快。

IV. 实现

我们的C++ 实现旨在尽可能快，同时保持通用性。我们通过为图中的顶点和边实现抽象基类来实现这一目标。这两个基类都提供了一套虚拟函数，便于用户子类化，而大多数内部操作则使用模板参数来实现效率。我们使用Eigen线性代数包 [11]，该包应用了SSE指令等优化技术，如惰性求值和循环展开，以实现高性能。

图3展示了我们系统的设计。要解决一个新的优化问题，只需要定义灰色框中的内容。使用提供的基类，派生一种新的节点类型只需要定义应用增量所需的 $\boxplus$算子。连接节点x_i和x $\boxplus$的边需要定义误差函数$\mathrm{e}_i()$。雅可比矩阵J随后会被数值计算，或者，为了更高的效率，用户可以通过重写虚拟基类函数显式指定J。因此，实现新的类型以解决新的优化问题或比较不同的参数化方法，只需要编写几行代码。

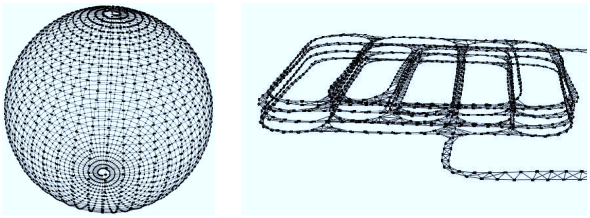


图4. 用于评估g²o的3D位姿图数据集：左侧图像显示一个模拟球体，右侧图像描绘了一个真实世界多级停车车库的数据集的部分视图。

并且 x_j 需要误差函数 $e_{ij}(\cdot)$ 的定义。雅可比矩阵 J_{ij} 然后被数值计算，或者，为了更高的效率，用户可以通过重写虚拟基类函数显式指定 J_{ij} 。因此，实现新的类型以解决新的优化问题或比较不同的参数化是一个编写几行代码的问题。

H的一般情况计算使用尺寸可变矩阵。如果变量的维度，即x_i的维度已知，我们的框架允许固定尺寸矩阵计算。利用先验已知的维度可以实现编译时优化，如循环展开，以执行矩阵乘法。

在实现Schur约化所需的矩阵乘法（见公式(25)）时已特别小心。利用底层图的结构，仅对形成H_{pp}额外条目的非零条目进行乘法。此外，我们在底层矩阵的块结构上操作（参见 [15]），与标量矩阵乘法相比，这导致更高效的缓存友好的矩阵乘法。

我们的框架对嵌入的线性求解器是无关的，因此我们可以为不同问题应用适当的求解器。我们使用了两种求解器进行实验。由于H是正半定和对称的，稀疏Cholesky分解产生一个高效的求解器 [4], [3]。请注意，在最小二乘迭代期间的非零模式是恒定的。因此，我们能够重用第一次迭代中计算的符号分解，这导致减少了填充并减少了后续迭代中的总计算时间。请注意，这种Cholesky分解没有利用参数的块结构。第二种方法是带块Jacobi预条件器的预条件共轭梯度（PCG） [13],，它在整个过程中利用块矩阵操作。由于PCG本身是一种迭代方法，求解线性系统需要 n 次迭代来处理 $n \times n$ 矩阵。由于执行 n 次PCG通常比Cholesky分解慢，我们将根据PCG的平方残差的相对减少来限制迭代次数。通过这种方式，我们能够量化由于提前终止PCG而引入的解精度损失。在实验中，我们将比较不同的求解器。

V. 实验

在本节中，我们展示了比较 g²o 与其他最先进优化方法的实验

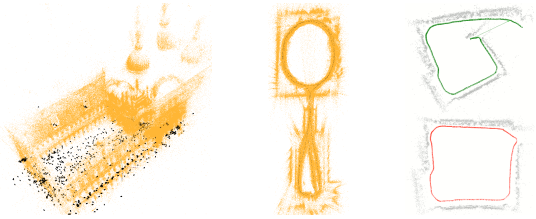


图5. 用于评估g²o的BA真实世界数据集和尺度漂移数据集：左侧图像显示威尼斯数据集，而中间图像描绘了新学院数据集 [24]。右侧的一对图像展示了经过单目SLAM处理的凯布尔学院数据集。此处发生尺度漂移（顶部），当使用7自由度相似变换闭合回路时即可纠正（底部）。

表I 测试数据集概述。

数据集	# 姿态	# 关键点	# 约束
Intel	943	-	1837
MIT	5489	-	7629
曼哈顿3500	3500	-	5598
维多利亚	6969	151	10608
Grid5000	5000	6412	82486
球体	2500	-	4949
车库	1661	-	6275
威尼斯	871	530304	2838740
新学院	3500	488141	2124449
尺度漂移	740	-	740

使用真实世界和合成数据集。图4描绘了 3D位姿图数据集，图5展示了 BA数据集和凯布尔学院的位姿图，该位姿图用于执行使用 7自由度相似性约束 [26],的尺度漂移感知 SLAM，图6显示了 2D数据集。每个数据集的变量和约束数量在表I中给出。所有实验都在一台以 2.8 GHz 运行的 Intel Core i7-930 的一个核心上执行。

我们比较 g²o 与其他最先进的实现：√SAM [5] 使用 M. Kaess 的开源实现，SPA [16],sSBA [15],和 RobotVision [26]。请注意，这些方法只针对优化问题的一个子集，而 g²o 能够解决所有这些问题，并且还可以轻松扩展到新问题。

A. 运行时间比较

在以下内容中，我们报告了每种方法执行一次迭代所需的时间。我们向每种方法提供了完整的优化问题，并执行了10次迭代，测量了每次迭代的平均耗时。在这组实验中，g²o使用CHOLMOD对线性系统应用Cholesky分解来求解，而我们比较的方法也使用了CHOLMOD。因此，所有方法求解线性系统所需的时间是相似的，差异反映了

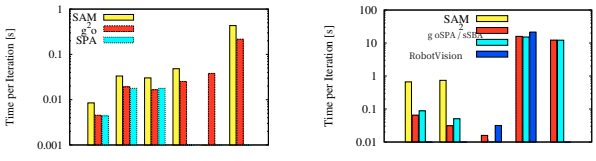


图7。每个数据集上每种方法的每次迭代时间。

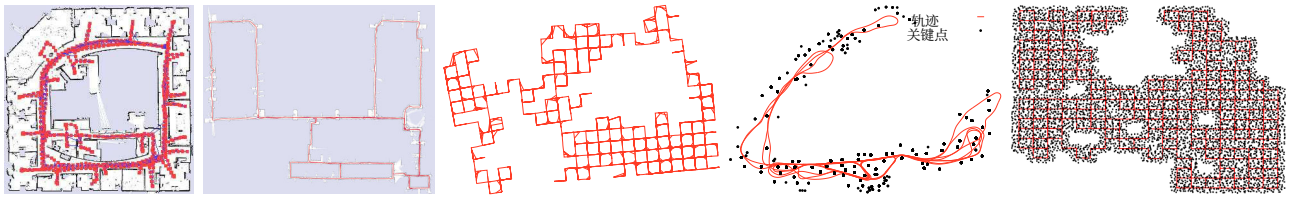


图6。用于评估g²o所使用的2D数据集。从左到右：英特尔研究实验室的2D位姿图；基利安法院的2D位姿图；曼哈顿3500，一个模拟位姿图；维多利亚公园带关键点的2D数据集；以及模拟关键点数据集Grid5000。

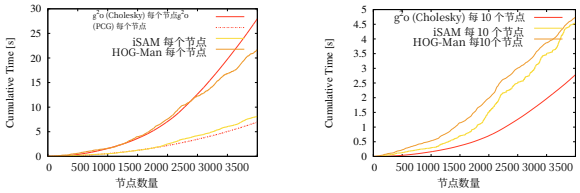


图8。曼哈顿3500数据集的在线处理。左图显示了在插入单个节点后进行优化的运行时间，而右图显示了在插入10个节点后进行优化的运行时间。

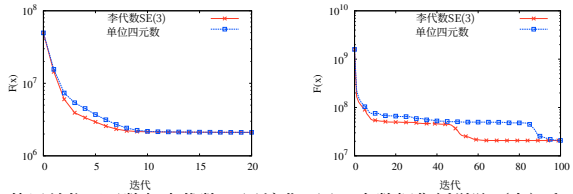


图9. 使用单位四元数与李代数se(3)演化F(x)，在数据集新学院（左）和威尼斯（右）上的表现。

构建线性系统的效率。结果汇总在图7中。

我们的系统 **g²o** 比我们在所有测试的2D和3D数据集上运行的SAM实现更快。虽然原则上它们实现了相同的算法， **g²o**利用高效的前端来生成线性化问题。在2D位姿图数据集上，我们框架的运行时间与高度优化的SPA实现相当。在BA数据集 **g²o** 实现了与sSBA相似的性能，sSBA比我们的通用框架稍快。与RobotVision相比， **g²o** 平均速度快两倍。

请注意，虽然g²o专注于批量优化，但可以通过在图中添加节点后进行优化来增量处理数据。g²o的效率产生了与为增量使用而设计的方法（如iSAM [14] 或HOG-Man [9]）相似的性能。如图8所示，通过每10个节点进行优化，或在插入单个节点后放宽PCG的终止标准，g²o可以实现可接受的运行时间。此外，它可以用作更复杂在线系统 [21], [9]的有效构建块。

如前所述，g²o可以数值计算雅可比矩阵J_{ij}，这允许快速原型化新的优化问题或新的误差函数。然而，通过解析指定雅可比矩阵，可以实现显著的加速。例如，当指定解析雅可比矩阵时，g²o在车库数据集上的一次迭代所需时间从80毫秒降至40毫秒。尽管效率有所降低，但在使用数值雅可比矩阵时，我们没有观察到精度下降。

B. 测试不同的参数化

由于我们的框架只需要实现误差函数和更新步骤，因此我们能够轻松比较不同的参数化方法。为此，我们为BA中的姿态表示实现了两种不同的参数化方法。在第一种参数化方法中，增量 Δx_i 由一个平移向量和单位四元数的轴表示。而在第二种方法中，增量 Δx_i 由李代数SE(3) [26]的成员表示。我们将不同的参数化方法应用于新学院数据集和威尼斯数据集。误差的演化如图9所示。两种参数化方法都收敛到相同的解，但使用SE(3)时收敛速度更快。

表II 不同线性求解器的比较（时间单位：秒）。

数据集	CHOLMOD	CSparse	PCG
Intel	0.0028	**0.0025**	0.0064 ± 0.0026
MIT	0.0086	**0.0077**	0.381 ± 0.364
曼哈顿3500	0.018	0.018	**0.011** ± **0**.0009
维多利亚	0.026	**0.023**	1.559 ± 0.683
Grid5000	**0.178**	0.484	1.996 ± 1.185
球体	0.055	0.398	**0.022** ± **0**.019
车库	0.019	0.032	**0.017** ± **0**.016
新学院	6.19	200.6	**0.778** ± **0**.201
威尼斯	1.86	39.1	**0.287** ± **0**.135
尺度漂移	0.0034	**0.0032**	0.005 ± 0.01

C. 线性求解器比较

我们的系统允许不同的线性求解器来求解公式(14)或公式(25)。目前我们实现了两个基于Cholesky分解的求解器，分别是CHOLMOD和CSparse [4]。此外，我们还实现了PCG，它是一种使用块-Jacobi预条件器的迭代方法。表II总结了在多个数据集上求解线性系统所需的时间。PCG在新学院和威尼斯数据集上表现非常好，比CHOLMOD快约7倍。PCG的收敛性取决于初始猜测值与最优解的接近程度。在实验中，如果相对残差低于给定阈值(10^{-8})，我们会终止PCG。因此，例如在MIT或维多利亚数据集上，PCG需要更多时间收敛。在较大的数据集上，CHOLMOD比CSparse快30倍。但令人惊讶的是，在较小的实例（如MIT数据集）上，CSparse是最快的求解器，它优于CHOLMOD和PCG。

D. 利用结构知识

如第三节-D所述，某些问题具有特征结构。利用这种结构可能显著提高线性系统的求解效率。

表III 不同线性求解器的比较。我们测量了g²o每次迭代的平均时间（单位：秒）。

数据集	直接求解 求解	舒尔分解 构建 / 求解 / 总计
维多利亚	0.026	0.029 / 0.121 / 0.150
Grid5000	0.18	0.12 / 0.16 / 0.28
新学院	15.18	3.37 / 7.07 / 10.44
威尼斯	33.87	11.25 / 1.78 / 13.03

基于地标的SLAM和BA具有相同的线性结构：关键点/点只能与机器人姿态/相机连接，导致黑塞矩阵H_{ll}的关键点部分呈现块对角结构。

在本实验中，我们评估了使用这种特定分解方法对基于关键点的SLAM和束调整的优势。表III显示了在不同数据集上启用和禁用分解的时间。从表中可以看出，当关键点数量超过位姿数量时，执行分解可以显著提高速度，这在束调整中通常是典型情况。然而，当位姿数量占主导地位时，执行Schur边缘化会导致高度连接的系统，其规模仅略有减小，并且需要更多计算资源来求解。

VI. 结论

在本文中，我们介绍了g²o，一个可扩展且高效的开放源代码框架，用于对可以嵌入图中的函数进行批量优化。属于此类的问题包括基于图的SLAM和束调整，这两个问题是机器人学和计算机视觉中两个基本且高度相关的问题。要使用g²o，只需定义误差函数和向当前解应用扰动的程序即可。此外，可以轻松地将新的线性求解器嵌入系统，从而验证特定求解器在共享这种图结构的各种问题中的特性。我们展示了g²o对各种SLAM变体（2D、3D、仅位姿和带关键点）和束调整的适用性。在大量数据集上进行的实际实验表明，g²o实现了与特定问题算法实现相当的性能，甚至经常优于它们。整个系统的开源实现作为ROS的一部分和OpenSLAM.org上免费提供。

致谢

我们感谢 E. Olson 提供的曼哈顿3500数据集，感谢 E. Nebot 和 H.Durrant-Whyte 提供的维多利亚公园数据集，感谢 N. Snavely 提供的威尼斯数据集，以及感谢 M. Kaess 提供的高效 iSAM 实现。

参考文献

[1] S. Agarwal、N. Snavely、S. M. Seitz 和 R. Szeliski，‘大规模束调整’，欧洲计算机视觉会议论文集（ECCV），2010。[2] M. Byrod 和 K. Astroem，‘共轭梯度束调整’，欧洲计算机视觉会议论文集（ECCV），2010。

[3] Y. Chen、T. A. Davis、W. W. Hager 和 S. Rajamanickam，‘算法 887：CHOLMOD、超节点稀疏 cholesky 分解和更新/下调整’，ACM 数学软件转，第 35 卷，第 3 期，第 1–14 页，2008。[4] T. A. Davis，《稀疏线性系统的直接方法》。SIAM，2006，SIAM 算法基础系列的一部分。[5] F. Dellaert 和 M. Kaess，‘平方根 SAM：通过平方根信息平滑进行的同时定位与地图构建’，国际机器人研究杂志，第 25 卷，第 12 期，第 1181–1204 页，2006 年 12 月。[6] T. Duckett、S. Marsland 和 J. Shapiro，‘快速、在线学习的全局一致地图’，自主机器人，第 12 卷，第 3 期，第 287 – 300 页，2002。[7] U. Frese，‘SLAM 信息矩阵近似稀疏性的证明’，IEEE 机器人与自动化国际会议论文集，2005。[8] U. Frese、P. Larsson 和 T. Duckett，‘多级松弛算法用于同时定位与地图构建’，IEEE 机器人转，第 21 卷，第 2 期，第 1–12 页，2005。[9] G. Grisetti、R. K¨ummerle、C. Stachniss、U. Frese 和 C. Hertzberg，‘流形上的分层优化用于在线 2d 和 3d 地图构建’，IEEE 机器人与自动化国际会议论文集，2010。[10] G. Grisetti、C. Stachniss 和 W. Burgard，‘非线性约束网络优化用于高效地图学习’，IEEE 智能交通系统转，2009。[11] G. Guennebaud、B. Jacob 等，‘Eigen v3’，http://eigen.tuxfamily.org，2010。[12] A. Howard、M. Matari´c 和 G. Sukhatme，‘网格上的松弛：广义定位的形式化’，IEEE/RSJ 智能机器人和系统国际会议论文集，2001。[13] Y. Jeong、D. Nister、D. Steedly、R. Szeliski 和 I. Kweon，‘突破现代束调整方法的极限’，IEEE 计算机视觉与模式识别会议论文集，2010。[14] M. Kaess、A. Ranganathan 和 F. Dellaert，‘iSAM：增量平滑与地图构建’，IEEE 机器人转，第 24 卷，第 6 期，第 1365–1378 页，2008 年 12 月。[15] K. Konolige，‘稀疏稀疏束调整’，英国机器视觉会议论文集，2010。[16] K. Konolige、G. Grisetti、R. K¨ummerle、W. Burgard、B. Limketkai 和 R. Vincent，‘2d 地图的稀疏姿态调整’，IEEE/RSJ 智能机器人和系统国际会议论文集，2010。¨K. Konolige，‘大规模地图构建’，美国人工智能会议论文集，2004。[18] M. A. Lourakis 和 A. Argyros，‘SBA：通用稀疏束调整软件包’，ACM 数学软件转，第 36 卷，第 1 期，第 1–30 页，2009。[19] F. Lu 和 E. Milios，‘全局一致的环境地图构建用范围扫描对齐’，自主机器人，第 4 卷，第 349 页，1997 333pp。 – 。[20] M. Montemerlo 和 S. Thrun，‘城市结构的规模化机器人 3d 地图构建’，国际实验机器人 symposium 论文集，2004。[21] K. Ni 和 F. Dellaert，‘使用嵌套分解的多级子图 SLAM’，IEEE/RSJ 智能机器人和系统国际会议论文集，2010。[22] E. Olson、J. Leonard 和 S. Teller，‘快速迭代优化具有差初始估计的姿态图’，IEEE 机器人与自动化国际会议论文集，2006，第 2262–2269 页。[23] W. Press、S. Teukolsky、W. Vetterling 和 B. Flannery，《数值食谱》，第二版。剑桥大学出版社，1992。[24] M. Smith、I. Baldwin、W. Churchill、R. Paul 和 P. Newman，‘新学院视觉和激光数据集’，国际机器人研究杂志，第 28 卷，第 5 期，第 595–599 页，2009 年 5 月。[25] R. Smith、M. Self 和 P. Cheeseman，‘机器人中不确定空间关系的估计’，Springer Verlag 编的《自主机器人车辆》，1990，第 167–193 页。[26] H. Strasdat、J. M. M. Montiel 和 A. Davison，‘尺度漂移感知的大规模单目 SLAM’，机器人：科学与系统会议论文集，2010。[27] B. Triggs、P. F. McLauchlan、R. I. Hartley 和 A. W. Fitzibbon，‘束调整——现代综合’，Springer Verlag 编的《视觉算法：理论与实践》（LNCS 系列论文集），2000，第 298–375 页。